

1 The §N anchor

Two grammars, one meaning: the template already anchors; §N catches what it does not.

```
(1) the A-grammar    ### A3 group . id A3.1    already says "division 3"
    (free)           ### P group . id P1      already says "page-wide"
```

```
(2) the §N anchor . POSITION is what makes it an anchor
    Aims    - [x] S1 the text ...    LEADS, after box, emoji, or id
    States  S1 shipped 260815 ...      LEADS the line
    Files   `build.py` S1 S2          TRAILS, and may be several
    Log     never anchored            it is a time axis
```

```
not anchors, all four seen on real pages
`QB6` S7 . [QB7 S3](...)    a division of ANOTHER page
S5.1 . S4.3                  a sub-division
SRequired Inputs             a named section, no number
"Every number S4 prints" a manuscript section, in ordinary prose
```

This part fixes the mark itself: what it points at, where it may appear, and what an unanchored line means. A page written to the current template needs NO anchor at all. Its Aims and States are already grouped **### A3** · and its ids already read **A3.1**, and both name division 3; **P1** already says page-wide. So the first grammar is free, and this page is its own proof: every one of its aims maps with zero anchors written. The **§N** anchor exists for what that grammar does not reach, which is every page written before it and every line that sits outside a group. The anchor is the literal token **§N**, where N is the number of a **### N** · division in this page's own Content. **POSITION** is what separates an anchor from prose, and it has to be. The **§** sign was in use on these boards long before this plugin, and a paper-section page writes **§4** in ordinary sentences to mean the manuscript's own section 4. No reading of the characters alone can tell those two apart, so the anchor is recognized only where prose does not put one: leading an Aim or State row, or trailing a Files row. An aim or state line carries at most one anchor, and it leads, after the checkbox, the state emoji, a decorative emoji, or the aim's id. A Files row carries several, and they trail at the end, because one code file often serves several divisions. Everything in the middle of a sentence is prose and is left alone. An unanchored line is legal and means page-wide: it belongs to the whole page, not to one division. So an existing page with no anchors at all parses cleanly today; its outline is simply one card, which is honest and uninformative. Log lines never take an anchor: the Log is ordered by time, and cutting it into divisions would cut its story. The anchor points by number, so whoever rennumbers Content divisions owes the page's anchors in the same edit; the parser catches an out-of-range **§N**, and it cannot catch an in-range one that now points at the wrong part.

2 The live surface: check and view in one walk

The tab's two jobs: the same parse measures the page and draws it.

```
header    [x] aligned . or 3 loose lines . S7 out of range
chips     [ By division] [ By progress]
lens      one card per division . under its title, ONE LINE saying
          what the division is about . then [x] done aims (each with
```

```

        its S-matched state receipt), then [ ] open aims . last
lens   [ ] open bucket FIRST (the page's to-do) then [x] done .
        every aim wears a div N badge linking back

one aim on a card . three parts, three jobs
  the goal    plain voice . the id ONCE, as a dim tag on the right
  NOW ...      the State beneath it, behind a `now` label
  the ask      a Decision Now row is neither, and gets its own card

the brief, read in this order and never invented
  the job line    ` This part fixes ...` . ` Establishes ...`
  else the caption `**The tab**`: the first surface renders ...`
  else the first plain sentence
[ ] else nothing      a wrong summary is worse than none

```

This part fixes the surface: rendered live on every open, never written to disk, wrong anchors shown rather than thrown. The view is rendered by GET `/_board/outline` on every open and stored nowhere, on the QP_{f1} precedent: a status written to disk starts aging the moment it lands. Because nothing is stored, this plugin has no staleness, no Regenerate, no subfolder, and no roster row: like it is a meta-surface over the page, the family's second. Checking is not a separate gate: an out-of-range SN renders as a named on the card it tried to reach, and every unanchored aim or state swells the card, so the card doubles as the worklist for anchoring the page. The two lenses are the same parsed data sorted twice, switched by chips client-side with no second request; a third lens later is one more chip, never a new writer. In the lens the [] bucket renders first, because opening that lens is asking what this page still owes. An Aim and its State are two different sentences, and the card has to keep them looking different. An Aim says what should become true; a State says what is true now. Printed as two alike lines that both open with the same id and both wear a tick, they read as one sentence said twice, and a reader stops trusting either. So the id prints once, as a dim tag on the right, the goal keeps the plain voice, and the status sits under it behind a small **now** label in the State's own emoji. A **Decision Now** row is a third thing again, an ask a person still owes, so it is neither counted as a state nor dropped: unanswered asks get their own card at the top, because a page waiting on someone should say so before it lists what it has done. A division card also says in one line what its division is about, and that line is read, never written. The template already asks each division for one: the line under its face diagram, saying what the part settles. Pages in the wild write the same line in their own hand, **Establishes the reading protocol**, so the reader looks for the SHAPE, an emoji then a sentence, rather than for one emoji. A division with no such line falls back to its figure caption, then to its first plain sentence, and a division with none of the three shows no brief at all. That last step is the same rule the anchors follow: a summary nobody wrote is not invented, because a wrong one costs the reader more than a missing one.

3 Migration: authored once, mechanical forever

Claude's one appearance: propose anchors into the .md, be reviewed, leave.

```

claude -p  reads page  -->  proposes SN per aim/state  -->  JL reviews
                                     |
forever after: the parser reads what the writer wrote  <-----+ written into .md

```

This part fixes how existing pages get their anchors without hand-writing sixty of them. Existing pages predate the anchor, so their outlines would open as one card each. The migration is a one-time pass per page: claude proposes an anchor for each aim and state line, the proposal is written into the `.md` as ordinary text, and a person reviews the diff before it lands. After that pass the judgment lives in the material and claude never renders, which is the whole point of the rule-based route. New pages anchor at write time: the writer who creates an aim knows its division better than any later reader.